

THE /goal COOKBOOK

Five ready-to-paste prompts that **audit, sharpen, revive, forge, and maintain** your Claude Code setup while you sleep.

COMPANION TO THE VIDEO: SELF-IMPROVING AGENTIC OS

Mark Kashef | Early AI Dopters

<https://www.skool.com/earlyaidopters/about>

HOW TO USE THIS COOKBOOK

Each of the 5 prompts inside is paste-ready. Open Claude Code, type **/goal**, paste the prompt body, hit enter. Claude runs turn-by-turn. After every turn, a small evaluator model checks the stop condition. When the condition is met the indicator clears and you wake up to a clean OS.

These prompts are not generic templates. They follow a 6-anchor structure that makes the difference between a /goal that finishes cleanly and a /goal that drifts. Learn the anchors below and you can write your own.

The 6 anchors of a great /goal prompt

- 1. Clear noun-target.** What folder, file, or system are you operating on? Be specific. *audit ~/.claude beats clean up my stuff.*
- 2. Per-item decision criteria.** For every item in scope, what is the rule that decides its fate? *used in the last 30 days OR load-bearing* is a rule. *looks important* is not.
- 3. A side-effect that actually changes state.** Move, write, archive, edit, delete. Without a state change there is nothing to verify. Saying *think about* is not a /goal, it is a chat.
- 4. A proof artifact.** A file that has to exist at the end. *AUDIT_REPORT.md*, *ALIVE.md*, *FORGE.md*. The evaluator can check a file. It cannot check a vibe.
- 5. A measurable stop condition.** Stop when X. X has to be checkable in one shell command. *under 20 skills* is checkable. *good enough* is not.
- 6. A turn-count safety net.** *Or stop after N turns.* Always include this. Without it a stuck /goal will burn tokens forever.

Read every prompt in this cookbook through the 6 anchors. Once you see them, you cannot unsee them.

STEP 0 — WARMUP: 60-SECOND HELLO-WORLD

When to use

- Your first time using /goal and you want to see the indicator clear without burning real work.
- You want to confirm the evaluator is set up before pasting one of the longer demo prompts.

The prompt — type /goal then paste this body

```
write a hello-world.py at the workspace root that prints "hello, self-improving OS" and the file exists. Or stop after 3 turns.
```

Guardrails

- Stop after 3 turns is the safety net. Lower it for warmups, higher for real work.
- The file path lives at the workspace root so verification is one ls away.

Wake-up check

```
$ ls hello-world.py  
$ cat hello-world.py
```

What good looks like

- hello-world.py exists at the workspace root.
- Running it prints "hello, self-improving OS".
- The /goal indicator at the bottom of the Claude Code prompt cleared.

STEP 1 — CLEAN: AUDIT YOUR ~/.CLAUDE

When to use

- Your ~/.claude/skills folder has grown past 20 entries.
- You haven't touched some skills in 30+ days but can't bring yourself to delete them.
- CLAUDE.md contradicts itself in places or references skills that no longer exist.
- You suspect skills overlap and need a senior-engineer-style audit pass.

The prompt — type /goal then paste this body

```
audit ~/.claude. For every skill in ~/.claude/skills/, decide if it earns its keep (referenced in a session in the last 30 days OR explicitly load-bearing for a workflow I run weekly). Archive the rest to ~/.claude/_attic/ with a timestamped subfolder. Walk every rule in .claude/rules/ and dedupe rules that say the same thing. If any two rules contradict each other, keep the more specific one and note the conflict in AUDIT_REPORT.md. Verify ~/.claude/CLAUDE.md references every kept skill by name. Write a manifest of every change to AUDIT_REPORT.md at the workspace root with: skills archived (and why), rules deduped (and which won), contradictions resolved, CLAUDE.md edits. Stop when skill count is under 20, AUDIT_REPORT.md exists and is non-empty, and a fresh check shows no two rules contradicting each other. Or stop after 25 turns and write a partial report.
```

Guardrails

- Always archive, never delete. The _attic folder is your safety net.
- Run with auto mode on or be ready to approve archive operations.
- If your skills folder is git-tracked, add: *If a path is git-tracked, use git mv instead of mv.*
- Strengthen the stop condition with *and AUDIT_REPORT.md is over 200 words* if the evaluator is too lenient.

Wake-up check

```
$ ls ~/.claude/skills | wc -l
$ ls ~/.claude/_attic/
$ head -50 AUDIT_REPORT.md
```

What good looks like

- Skill count under 20.
- _attic folder contains every archived skill, organized by date.
- AUDIT_REPORT.md reads like a senior engineer's PR description, ~300-500 words.

- CLAUDE.md is shorter and every reference is current.

STEP 2 — SHARPEN: ITERATE ONE SKILL AGAINST A RUBRIC

When to use

- You have a skill that works but you suspect it could be sharper.
- You can write a small rubric (5 criteria, each pass/fail) and a handful of test inputs.
- You want measurable improvement, not vibe-improvement.

The prompt — type `/goal` then paste this body

```
iterate on .claude/skills/thumbnail-prompt-builder/SKILL.md. For each input in
test_inputs.md, simulate the skill's output, then score it against rubric.md.
Log every run to ITERATION_LOG.md with score and what you will change next. Edit
SKILL.md to fix the lowest-scoring criterion without regressing on previously
passing criteria. Re-run all 5 tests. Repeat. Stop when 5 of 5 tests score 5 out
of 5 and ITERATION_LOG.md shows the full progression with at least 6 runs. Or
stop after 15 turns.
```

Guardrails

- Write `rubric.md` and `test_inputs.md` FIRST. Without them this `/goal` has nothing to optimize against.
- The *without regressing on previously passing criteria* clause is what prevents Claude from over-fitting.
- `ITERATION_LOG.md` is the proof. Read it after to see exactly what changed run by run.
- If runs feel repetitive, tighten the rubric. Each criterion should be unambiguously pass or fail.

Wake-up check

```
$ tail -100 ITERATION_LOG.md
$ git diff .claude/skills/thumbnail-prompt-builder/SKILL.md
```

What good looks like

- Run 1 scored 2 or 3 out of 5. Final run scores 5 of 5.
- `ITERATION_LOG.md` shows a clean progression of scores climbing.
- The `SKILL.md` diff is surgical, not a full rewrite.
- Re-running tests by hand on the new `SKILL.md` still scores 5 of 5.

STEP 3 — REVIVE: RESURRECT YOUR PROJECT BACKLOG

When to use

- You have a ~/Projects folder full of half-finished work you no longer remember.
- You want a one-line verdict on every dormant folder without opening any of them.
- You'd rather know what's worth reviving than feel guilty about all of it.

The prompt — type /goal then paste this body

```
scan ~/Projects for every subfolder. For each folder, check git log -1. If the latest commit is over 90 days old OR there is no .git folder, treat it as dormant. For each dormant folder, attempt to run it, if it fails fix dependencies to 2026 standards, rewrite the README in 3 sentences (what it does, how to run, what is special), log a verdict (alive / revived / unfixable) to ALIVE.md with a one-line reason. Do not commit. Stop when every dormant folder has a verdict line in ALIVE.md. Or stop after 30 turns.
```

Guardrails

- The *do not commit* clause is critical. You want the diagnostic, not a flood of auto-commits.
- Tune the 90-day window to match your sense of dormant. 180 days is fine for slower-moving projects.
- If you have hundreds of folders, raise the turn cap to 60 or run in batches.
- *Unfixable* is a real verdict. Not every old project deserves resurrection.

Wake-up check

```
$ cat ALIVE.md
$ grep '^- revived' ALIVE.md | wc -l
```

What good looks like

- ALIVE.md has one verdict line per dormant folder.
- Verdicts split roughly: a third alive-already, a third revived, a third unfixable.
- Revived folders have a fresh 3-sentence README you can actually read.
- You can decide what to keep without opening any of the folders yourself.

STEP 4 — FORGE: GENERATE SKILLS FROM YOUR OWN SESSION HISTORY

When to use

- You've used Claude Code enough that ~/.claude/projects/ has dozens of session transcripts.
- You suspect you're typing the same kinds of prompts over and over.
- You want Claude to spot those patterns and turn them into skills automatically.

The prompt — type /goal then paste this body

```
read the 50 most recent session transcripts in ~/.claude/projects/. Find 3
recurring patterns of user prompts that appear 5 or more times and do NOT have a
corresponding skill in ~/.claude/skills/. For each pattern, write a new skill
folder at ~/.claude/skills/SKILL_NAME/ with SKILL.md (proper YAML frontmatter —
name, description, when-to-trigger) and a SMOKE_TEST.md showing one example. Log
each to FORGE.md with the pattern detected and a count of sessions it would have
helped. Stop when 3 new skill folders exist, each contains SKILL.md and
SMOKE_TEST.md, and FORGE.md is non-empty. Or stop after 25 turns.
```

Guardrails

- Tighten *5 or more times* if the agent is too eager. Bump to 8 or 10 for stricter pattern detection.
- SMOKE_TEST.md per skill is what stops Claude from generating skill folders that don't actually run.
- Review every generated skill before relying on it. The forge produces drafts, not finished work.
- Pair this with the Clean prompt on a heartbeat — Clean prunes, Forge generates, equilibrium holds.

Wake-up check

```
$ ls ~/.claude/skills | wc -l
$ cat FORGE.md
$ ls ~/.claude/skills/SKILL_NAME/ # for each new skill
```

What good looks like

- 3 new skill folders exist, each with SKILL.md (valid YAML) and SMOKE_TEST.md.
- FORGE.md lists each pattern detected with a count of sessions it would have helped.
- Each new skill maps to a real recurring task you actually do.
- Running the SMOKE_TEST.md example end-to-end succeeds for each new skill.

STEP 5 — MAINTAIN: /LOOP + /GOAL HEARTBEAT

When to use

- You want the OS to stay clean by itself between recording sessions, audits, and forge runs.
- You're comfortable letting Claude Code keep a long-running cycle going while you do other work.
- You have a maintenance log file you want appended to so future-you can read the trail.

The prompt — type /goal then paste this body

```
/loop 30m /goal keep ~/.claude clean. Archive any skill not referenced in 30 days to ~/.claude/_attic/. Refresh ~/.claude/CLAUDE.md to reference every active skill and drop references to anything no longer present. Resolve any new rule contradictions by keeping the more specific. Append a paragraph to MAINTENANCE_LOG.md with timestamp and what changed. Stop the cycle when MAINTENANCE_LOG.md has a new entry and no contradictions remain. Or stop after 8 turns per cycle.
```

This one starts with /loop, not /goal. /loop fires the /goal command on a recurring timer. Both /loop and /goal stay live until you /exit.

Guardrails

- /loop fires the /goal on the interval you give it (here 30m). Both stay live until you /exit.
- Tune the 30m cadence to your usage. 6h is fine for slow weeks, 15m for active build sessions.
- Cap turns per cycle low (8 is conservative) so cycles end fast and don't blow your context.
- If MAINTENANCE_LOG.md grows past a few hundred lines, rotate it. The agent can do that too.

Wake-up check

```
$ tail -60 MAINTENANCE_LOG.md  
$ ls ~/.claude/skills | wc -l # should stay stable cycle-over-cycle
```

What good looks like

- MAINTENANCE_LOG.md gains a new dated paragraph per cycle.
- Skill count stays in your target range without manual intervention.
- No new rule contradictions accumulate.
- After several cycles the change-per-cycle diff is small — equilibrium reached.

BUILD YOUR OWN /goal

Use this template to write your own /goal prompts. Each blank corresponds to one of the 6 anchors.

```
[VERB] [NOUN-TARGET]. For every [ITEM] in [SCOPE], decide if it [CRITERIA].  
[SIDE-EFFECT for the items that pass]. [SIDE-EFFECT for the items that don't  
pass]. Log every decision to [PROOF-ARTIFACT.md] at the workspace root with  
[WHAT TO RECORD]. Stop when [MEASURABLE CONDITION 1] AND [MEASURABLE CONDITION  
2]. Or stop after [N] turns and write a partial report.
```

Worked example — fill in the blanks

```
audit ~/Downloads. For every file in ~/Downloads older than 90 days, decide if  
it is a screenshot, an installer, or content I might still want. Move  
screenshots and installers to ~/Downloads/_attic/. Leave content files in place.  
Log every decision to DOWNLOADS_AUDIT.md at the workspace root with filename,  
age in days, decision, and reason. Stop when no files older than 90 days remain  
in ~/Downloads (excluding _attic) AND DOWNLOADS_AUDIT.md is over 50 lines. Or  
stop after 20 turns and write a partial report.
```

Notice the 6 anchors are all present. The noun is ~/Downloads. The criteria is the file-type rule. The side-effects are move-to-attic. The proof is DOWNLOADS_AUDIT.md. The stop conditions are checkable in one shell command. The safety net is 20 turns.

Want to write /goal prompts like this for your own workflow?

Inside the Early AI Dopters community, you get the full **LIVING** Claude Code Magic course:

- The /goal and /loop primitives taught from zero, with the 6 anchors as a working framework.
- The exact prompts I run on my own OS, walked through line by line in a dedicated module.
- Direct access to me and my team of coaches when you hit a wall.
- A working group of builders shipping their own /goal cookbooks.

JOIN THE COMMUNITY

<https://www.skool.com/earlyaidopters/about>

Companion to the YouTube video **Self-Improving Agentic OS**. Drop questions in the comments or in Skool.
Build something. Ship it.